Recursion

The Pattern

- when a method calls itself
- Classic example--the factorial function:
  - $n! = 1 \cdot 2 \cdot 3 \cdot \cdots \cdot (n-1) \cdot n$
- Recursive definition:

$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot f(n-1) & \text{else} \end{cases}$$

As a Python method:

```
1  def factorial(n):
2      if n == 0:
3          return 1
4      else:
5          return n * factorial(n-1)
```

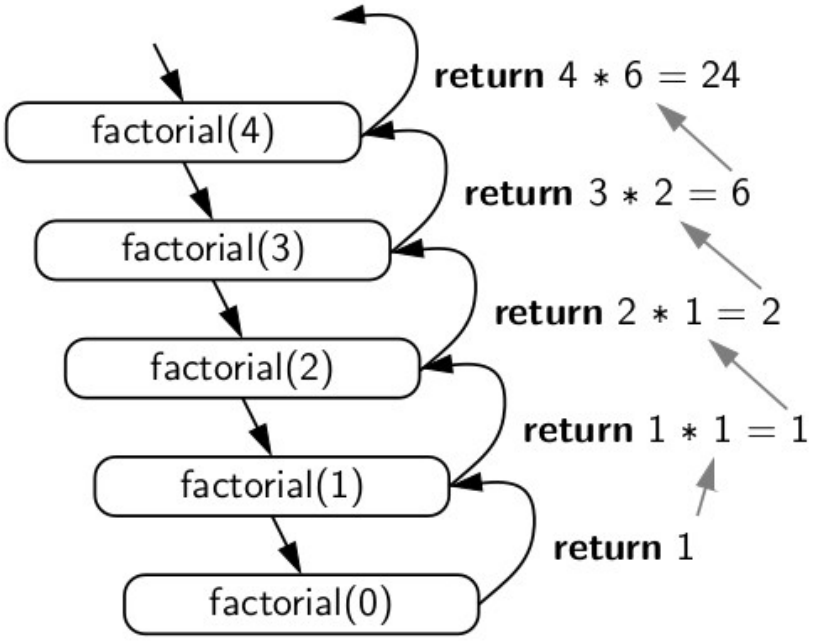
Content of a Recursive Method

- Base case(s)
  - Values of the input variables for which we perform no recursive calls are called base cases (there should be at least one base case).
  - Every possible chain of recursive calls must eventually reach a base case.
- Recursive calls
  - Calls to the current method.
  - Each recursive call should be defined so that it makes progress towards a base case.

Visualizing

- trace
  - A box for each recursive call
  - An arrow from each caller to callee
  - An arrow from each callee to caller showing return value

- Example

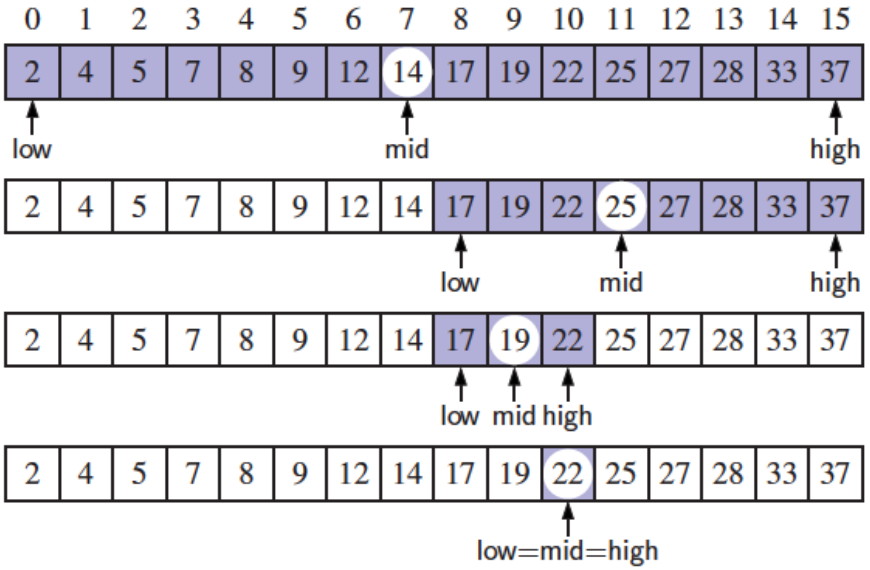


Recursive binary search

```
def bin_search(arr, key, low=0, high=None):

  if high is None:
    high = len(arr) - 1

  if low > high:
    return -1

  mid = low + (high - low) // 2

  if arr[mid] == key:
    return mid
  elif  key < arr[mid]:
    return bin_search(arr, key, low, mid - 1)
  else:
    return bin_search(arr, key, mid + 1, high)


arr = [12, 51, 14, 123, 80, 78, 90, 178]
arr.sort()
x = 14

status = bin_search(arr, x)
```

Visualizing Binary Search

- We consider three cases:
  - If the target equals data[mid], then we have found the target.
  - If target $<$ data[mid], then we recur on the first half of the sequence.
  - If target $>$ data[mid], then we recur on the second half of the sequence.



Computing Powers

- The power function, $p(x,n)=x^n$, can be defined recursively:

$$p(x,n)=\begin{cases} 1 & \text{if } n=0 \\ x\cdot p(x,n-1) & \text{else} \end{cases}$$

- This leads to an power function that runs in O(n) time (for we make n recursive calls).
- We can do better than this, however.

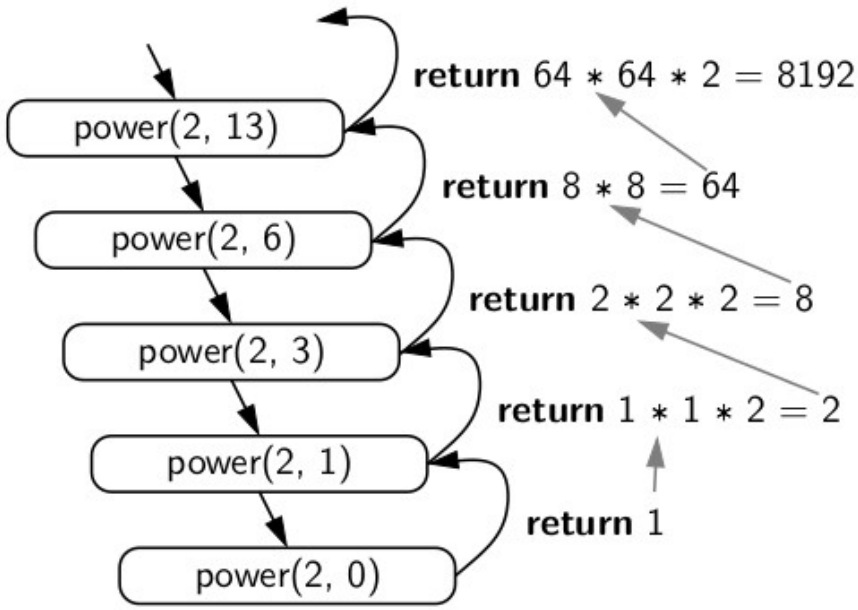
Recursive Squaring

- We can derive a more efficient linearly recursive algorithm by using repeated squaring:

$$p(x,n)=\begin{cases} 1 & \text{if } x=0 \\ x\cdot p\left(x,(n-1)/2\right)^2 & \text{if } x>0 \text{ is odd} \\ p\left(x,n/2\right)^2 & \text{if } x>0 \text{ is even} \end{cases}$$

Recursive Squaring Method

```
1  def power(x, n):
2      """Compute the value x**n
3      if n == 0:
4          return 1
5      else:
6          partial = power(x, n // 2)
7          result = partial * partial
8          if n % 2 == 1:
9              result *= x
10         return result
```



Reversing an Array

Algorithm ReverseArray(A, i, j):

Input: An array A and nonnegative integer indices i and j

Output: The reversal of the elements in A starting at index i and ending at j

Defining Arguments for

- In creating recursive methods, it is important to define the methods in ways that facilitate .
- For example, we defined the array reversal method as ReverseArray(A, i, j), not ReverseArray(A).

Python version:

```
1  def reverse(S, start, stop):
2    """Reverse elements in implicit slice S[start:stop]."""
3    if start < stop - 1:                          # if at least 2 elements:
4      S[start], S[stop-1] = S[stop-1], S[start]   # swap first and last
5      reverse(S, start+1, stop-1)                 # recur on rest
```

Tail Recursion

- Tail occurs when a linearly recursive method makes its recursive call as its last step. Eg. array reversal seen earlier.
- Such methods can be easily converted to non-recursive methods (which saves on some resources).

Algorithm IterativeReverseArray(A, i, j):

Input: An array A and nonnegative integer indices i and j

Output: The reversal of the elements in A starting at index i and ending at j

while $i < j$ **do**

Swap $A[i]$ and $A[j]$

$i = i + 1$

$j = j$ - 1

return